THE UNIVERSITY OF AZAD JAMMU
AND KASHMIR, MUZAFFARABAD

LAB 02

FROM REAL WORLD CONCEPT TO SOFTWARE OBJECT

(BANK MANAGEMENT SYSTEM)

SUBMISSION DETAILS	
Student Name	**Misbah Khurshid**
Roll Number	**2025- SE -10**
Course Code	**SEC-532**
Course Name	**Software Design & Architecture**
Lab Title	**From Real World Concept to Software Object (Bank Management System)**
University	**The University of Azad Jammu and Kashmir**

Academic Year 2025–2029 | Software Design & Architecture | 2025-SE-10

Bank Management System: Conceptual Classes and Abstraction

1. Introduction and Selected Domain:

For this assignment, I selected the Bank Management System as my real-world domain. A bank manages many activities, such as customer accounts, deposits, withdrawals, money transfers, transactions, loans, and customer information. When these activities are handled without a proper structure, errors may occur and customer records can become difficult to manage. The purpose of this system is to organize banking activities in one place so that daily operations become easier, faster, and more reliable.

2. Candidate Conceptual Classes:

The following 10 conceptual classes represent meaningful entities or ideas in the bank management domain. They are not screens, database tables, or pieces of code.

1. **Customer:** A person who uses the bank's services. It stores personal and contact information.
2. **BankAccount:** A customer's account maintained by the bank. It stores the account number, account type, and balance.
3. **Transaction:** A record of financial activity, such as a deposit, withdrawal, or transfer.
4. **Deposit:** Money added to a customer's bank account.
5. **Withdrawal:** Money taken out of a customer's bank account.
6. **FundTransfer:** A movement of money from one bank account to another.
7. **Loan:** Money borrowed by a customer from the bank under agreed conditions.
8. **Employee:** A bank staff member who helps customers and performs authorized banking tasks.
9. **DebitCard:** A card linked to a bank account that can be used for payments or cash withdrawals.
10. **Beneficiary:** A saved recipient who can receive money through a transfer.

3. Attributes and Methods of Three Important Classes

Attributes are the information a class stores, while methods are the actions or behavior it can perform. The three selected classes are Customer, BankAccount, and Transaction.

3.1 Customer Class

Attributes:

• **customerId:** A unique identifier for each customer.
• **fullName:** The customer's complete name.
• **phoneNumber:** The customer's contact number.
• **email:** The customer's email address.
• **address:** The customer's residential address.

Methods

• **updateProfile():** Updates the customer's personal details.
• **openAccount():** Requests or creates a new bank account.
• **viewAccounts():** Displays accounts associated with the customer.
• **viewTransactionHistory():** Displays the customer's transaction records.

3.2 BankAccount Class

Attributes:

• **accountNumber:** A unique number used to identify the account.
• **accountType:** Identifies the account type, such as savings or current.
• **balance:** Stores the available amount of money in the account.

• **openingDate:** Records the date the account was opened.
• **status:** Shows whether the account is active, blocked, or closed.

Methods:
• **deposit(amount):** Adds money to the account.
• **withdraw(amount):** Removes money if the available balance is sufficient.
• **checkBalance():** Returns the current account balance.
• **closeAccount():** Closes the account when permitted.

3.3 Transaction Class:

Attributes:
• **transactionId:** A unique identifier for a transaction.
• **transactionType:** Identifies the type, such as deposit, withdrawal, or transfer.
• **amount:** Stores the amount involved in the transaction.
• **transactionDate:** Records when the transaction occurred.
• **status:** Shows whether the transaction is pending, completed, or failed.

Methods:
• **createTransaction():** Creates a transaction record.
• **validateTransaction():** Checks that the transaction details are valid.
• **processTransaction():** Processes an authorized transaction.
• **generateReceipt():** Produces a receipt containing transaction details.

4. Main Abstraction Decisions

Customer, BankAccount, and Transaction were selected for detailed modeling because they are central to everyday banking operations. The other classes support activities such as deposits, withdrawals, transfers, loans, and card services. Advanced features such as fraud detection, foreign exchange, investment banking, complex loan-risk calculations, and regulatory reporting are excluded because they are beyond the scope of this small conceptual model. Screens, database tables, login controllers, and programming-language details are also excluded because conceptual modeling focuses on real-world banking concepts rather than technical implementation.

5. Invented Software Object

Transaction Validator:

A Transaction Validator is an invented software object. It does not represent a physical entity in a bank, but it is useful for enforcing transaction rules.

Purpose: Checks a transaction before it is processed.

Validation: Checks that the amount is greater than zero, the account exists and is active, and sufficient funds are available for a withdrawal or transfer.

Result: Returns whether the transaction can proceed or explains why it was rejected.

Benefit: Helps prevent invalid transactions and keeps validation rules consistent.

Although it has no direct real-world counterpart, this object demonstrates that software design may require objects created to coordinate processes or enforce business rules.

6. Conclusion:

The Bank Management System is a practical example of conceptual modeling. By identifying conceptual classes, defining their attributes and methods, and making clear abstraction decisions, the system can be organized around real banking needs before implementation begins.